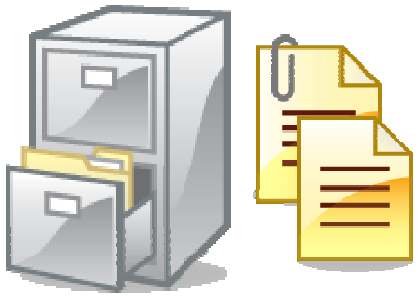
Obiektowe bazy danych

Plan rozdziału

- Wprowadzenie
  - obiektowy model danych
- Obiekty w bazie danych
  - definiowanie i przechowywanie
  - składowe i metody
  - konstruktory
  - referencje
  - dziedziczenie i polimorfizm
- Kolekcje
  - tablice o zmiennym rozmiarze
  - tablice zagnieżdżone
- Perspektywy obiektowe

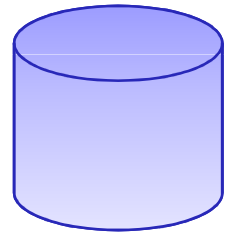
Model danych

- Model danych to abstrakcyjny opis sposobu **reprezentacji** i **wykorzystania** danych
- Na model danych składają się:
  - **struktura**: kolekcja struktur wykorzystywanych do przedstawiania obiektów i encji modelowanego mini-świata,
  - **ograniczenia**: reguły zapewniające spójność i poprawność danych
  - **operacje**: zbiór operatorów stosowalnych do struktur w celu czytania i pisania danych



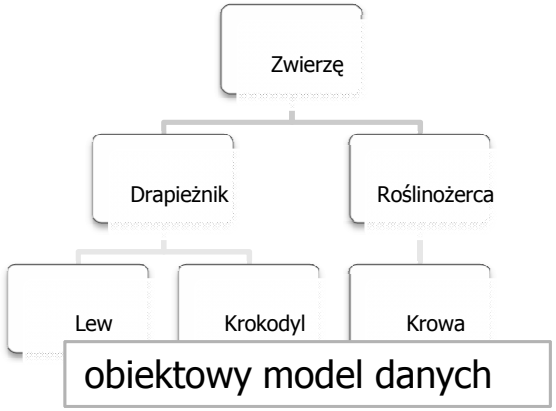
ID_PRAC	NAZWISKO	ETAT	ID_SZEFA	ZATRUDNI	PLACA_POD	PLACA_DOD	ID_ZESP
100	WEGLARZ	DYREKTOR		68/01/01	1730	420,5	10
110	BLAZEWICZ	PROFESOR	100	73/05/01	1350	210	40
120	SLOWINSKI	PROFESOR	100	77/09/01	1070		30
130	BRZEZINSKI	PROFESOR	100	68/07/01	960		20
140	MORZY	PROFESOR	130	75/09/15	830	105	20
150	KROLIKOWSKI	ADIUNKT	130	77/09/01	645,5		20
160	KOSZLAJDA	ADIUNKT	130	85/03/01	590		20
170	JEZIERSKI	ASYSTENT	130	92/10/01	439,7	80,5	20
190	MATYSIAK	ASYSTENT	140	93/09/01	371		20
180	MAREK	SEKRETARKA	100	85/02/20	410,2		10
200	ZAKRZEWICZ	STAZYSTA	140	94/07/15	208		30
210	BIALY	STAZYSTA	130	93/10/15	250	170,6	30
220	KONOPKA	ASYSTENT	110	93/10/01	480		20
230	HAPKE					90	30

relacyjny model danych



baza danych





obiektowy model danych



aplikacja

Obiektowy model danych

- Pojęcia podstawowe
  - **klasa**: abstrakcyjna reprezentacja "rzeczy", zawiera definicję struktury i zachowania w postaci składowych i metod
  - **obiekt**: wystąpienie klasy, reprezentacja konkretnego bytu
  - **metoda**: zdolność obiektu do wykonania czynności
  - **przekazywanie komunikatów**: sposób przekazywania danych lub wywoływania czynności między obiektami
  - **dziedziczenie**: uproszczony sposób specjalizacji klas
  - **enkapsulacja**: ukrywanie złożoności obiektu za interfejsem
  - **polimorfizm**: zdolność występowania obiektów w kontekście ich klas-przodków

Zalety modelu obiektowego

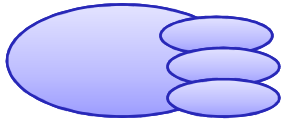
- Modelowanie
  - naturalna reprezentacja złożonych obiektów świata rzeczywistego
  - naturalna reprezentacja relacji kompozycji
  - naturalna reprezentacja interakcji między obiektami
  - bezpośrednie powiązanie operacji z danymi
- Programowanie
  - wielokrotne użycie kodu
  - modularność i łatwość pielęgnacji
  - zwiększone bezpieczeństwo i elastyczność
  - brak konieczności konwersji między modelami danych na styku baza danych – aplikacja (ang. *O/R impedance mismatch*)

Model obiektowo-relacyjny

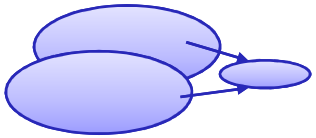
- Umożliwia wykorzystanie mechanizmów obiektowych wewnątrz relacyjnej bazy danych
  - abstrakcyjne typy danych definiowane przez użytkownika
  - predefiniowane typy obiektowe dostarczane przez producentów oprogramowania (multimedia, dane przestrzenne)
  - dziedziczenie, enkapsulacja, polimorfizm, klasy abstrakcyjne, …
  - dostępne mechanizmy relacyjne: SQL, przetwarzanie transakcyjne, zarządzanie współbieżnością, ścieżki dostępu, kopie bezpieczeństwa i protokoły odtwarzania spójności, …
- Alternatywa dla systemów odwzorowania obiektowo-relacyjnego (O/RM)

Własności obiektowo-relacyjne w Oracle

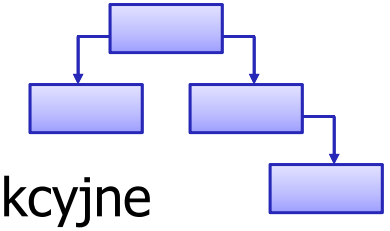
- Obiektowe typy danych użytkownika
  - metody, składowe, konstruktory, przeciążanie



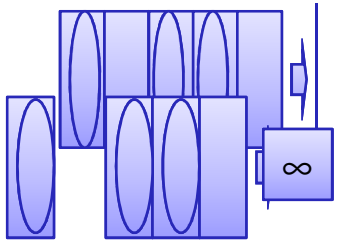
- Współdzielenie obiektów
  - referencje, nawigacja



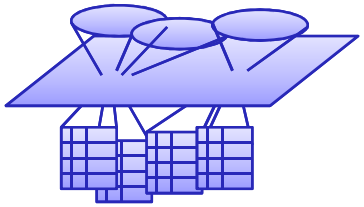
- Dziedziczenie
  - polimorfizm, przesłanianie metod, typy abstrakcyjne



- Kolekcje
  - tabele o zmiennym rozmiarze, tabele zagnieżdżone

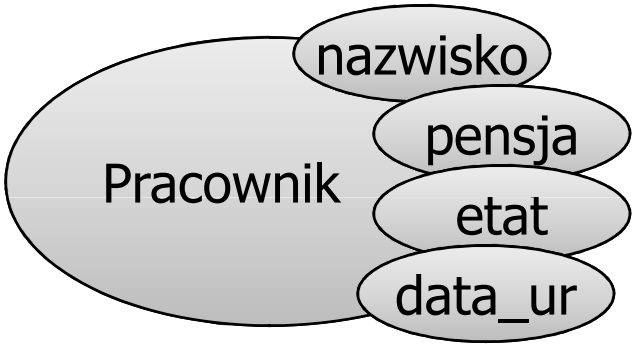


- Perspektywy obiektowe



Abstrakcyjne typy danych użytkownika

- Każdy typ obiektowy składa się z dwóch części:
  - **deklaracji** (interfejsu): zbioru składowych (pól i sygnatur metod)
  - **definicji** (ciała): implementacji metod



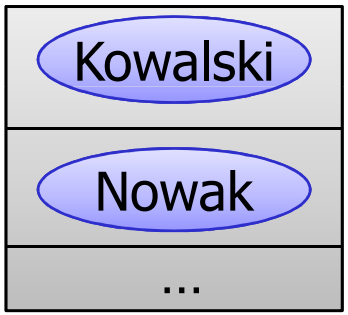

```
CREATE TYPE Pracownik AS OBJECT (
  nazwisko VARCHAR2(20),
  pensja   NUMBER(6,2),
  etat     VARCHAR2(15),
  data_ur  DATE
);
```

Trwałość obiektów

- Obiekty składuje się trwale w w dwóch postaciach:
  - **obiekty krotkowe** (ang. *row object*)
  - **obiekty atrybutowe** (ang. column object)

- Do przechowywania obiektów krotkowych wykorzystuje się **tabele obiektów** (ang. *object table*)

`PracownicyObjTab`



- Obiekty atrybutowe są przechowywane jako wartości atrybutu w zwyczajnej tabeli

`ProjektyTab`

`symbol`		`budzet`	`kierownik`
AB 001	...	20 000	Kowalski
XY 222	...	15 000	Nowak

Tworzenie obiektów

- Do tworzenia obiektu służy specjalna metoda – **konstruktor**
- Każdy typ obiektowy posiada **konstruktor atrybutowy**
  - lista argumentów zgodna z listą atrybutów typu obiektowego
  - podczas wywołania należy podać wartości wszystkich atrybutów
- Nazwa konstruktora jest identyczna z nazwą typu obiektowego
- Wywołanie konstruktora poprzedzamy słowem **`NEW`**
- Użytkownik może deklarować własne konstruktory
- Sposób tworzenia obiektu jest identyczny dla obiektów krotkowych i obiektów atrybutowych

Obiekty krotkowe

- Utworzenie tabeli obiektów

```
CREATE TABLE PracownicyObjTab
OF Pracownik;
```

- Wstawienie obiektu krotkowego

```
INSERT INTO PracownicyObjTab VALUES
(NEW Pracownik('Kowalski',2500,'ASYSTENT',
                DATE '1965-07-01'));
```

- Dostęp obiektowy i relacyjny do obiektów krotkowych

```
SELECT VALUE(p) FROM PracownicyObjTab p;

SELECT * FROM PracownicyObjTab;
```

Obiekty atrybutowe

- Utworzenie tabeli z obiektami atrybutowymi

```
CREATE TABLE ProjektyTab (
  symbol CHAR(6), nazwa VARCHAR(100),
  budzet NUMBER, kierownik Pracownik);
```

- Wstawienie obiektu atrybutowego

```
INSERT INTO ProjektyTab VALUES
('AB 001','Projekt X',20000,
  NEW Pracownik('Nowak',3200,'ADIUNKT',null));
```

- Dostęp obiektów atrybutowych i ich składowych

```
SELECT nazwa, kierownik FROM ProjektyTab;

SELECT p.kierownik.nazwisko FROM ProjektyTab p;
```

alias jest wymagany w przypadku dostępu do składowych obiektu atrybutowego

Definiowanie metod typu obiektowego

- Typ obiektowy może posiadać następujące rodzaje metod
  - **`MEMBER`**: metody wołane na rzecz konkretnego obiektu
  - **`STATIC`**: metody wołane na rzecz całego typu obiektowego
  - **`MAP/ORDER`**: metody wykorzystywane do porównywania obiektów
  - **`CONSTRUCTOR`**: metody tworzące nowe obiekty

- Metoda jest definiowana dwukrotnie
  - w deklaracji typu: **sygnatura metody** (nazwa i lista argumentów)
  - w definicji typu: **ciało metody** (implementacja, kod źródłowy)

Definiowanie metod typu obiektowego

```
ALTER TYPE Pracownik REPLACE AS OBJECT (
  nazwisko VARCHAR2(20),
  pensja   NUMBER(6,2),
  etat     VARCHAR2(15),
  data_ur  DATE,
  MEMBER FUNCTION wiek RETURN NUMBER,
  MEMBER PROCEDURE podwyzka(p_kwota NUMBER) );
```

```
CREATE OR REPLACE TYPE BODY Pracownik AS
  MEMBER FUNCTION wiek RETURN NUMBER IS
  BEGIN
    RETURN EXTRACT (YEAR FROM CURRENT_DATE) –
           EXTRACT (YEAR FROM data_ur);
  END wiek;

  MEMBER PROCEDURE podwyzka(p_kwota NUMBER) IS
  BEGIN
    pensja := pensja + p_kwota;
  END podwyzka;
END;
```

Aktywowanie metod składowych

- Metody składowe mogą być uruchamiane:
  - z poziomu SQL (tylko metody będące funkcjami)
  - z poziomu PL/SQL (wszystkie metody)

```
SELECT p.nazwisko, p.data_ur, p.wiek() FROM PracownicyObjTab p;
```

```
DECLARE
  l_kierownik Pracownik;
BEGIN
  SELECT kierownik INTO l_kierownik
  FROM ProjektyTab WHERE symbol = 'AB 001';

  l_kierownik.podwyzka(200);

  UPDATE ProjektyTab
  SET kierownik = l_kierownik
  WHERE symbol = 'AB 001';
END;
```

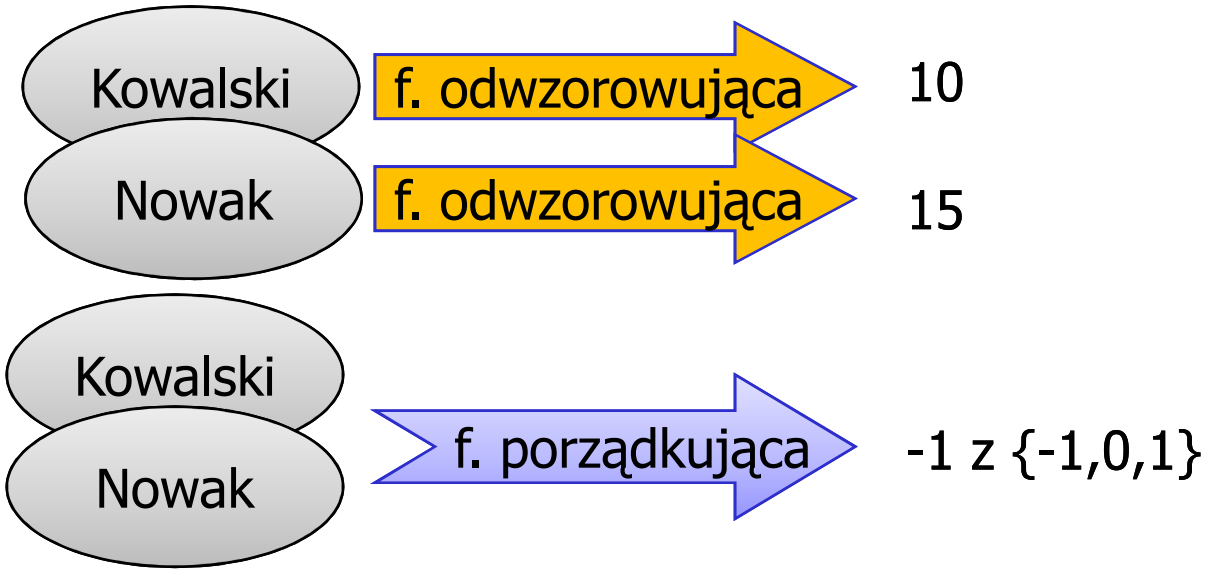
Tożsamość obiektów

- Tożsamość obiektu to unikalny i niezmienny identyfikator związany z danym obiektem przez cały cykl jego życia
  - tożsamość posiadają tylko obiekty krotkowe
  - tożsamość jest zapewniana przez OID (ang. *object identifier*)
  - identyfikatory są lokalnie i globalnie unikalne, automatycznie indeksowane i zajmują 16B; można również wykorzystać klucz podstawowy tabeli obiektowej do generowania identyfikatorów OID
  - aplikacja może wykorzystywać referencje do identyfikatorów, odczytywane za pomocą funkcji **REF( )**

```
SELECT VALUE(p), REF(p) FROM PracownicyObjTab p;
```

Porównywanie wartości obiektów

- Wartością obiektu jest zbiór wartości jego składowych
  - dwa obiekty są równe gdy mają te same wartości składowych
  - standardowo, operatory = i != umożliwiają porównywanie obiektów
- Porównywanie obiektów najczęściej wymaga sortowania
  - operatory relacyjne: <, >, <=, >=,
  - operator **`BETWEEN ... AND ...`**,
  - klauzule **`ORDER BY`**, **`GROUP BY`**, **`DISTINCT`**
  - porównywanie za pomocą funkcji odwzorowującej lub porządkującej



Metoda odwzorowująca

- Dodanie sygnatury metody do deklaracji typu obiektowego

```
ALTER TYPE Pracownik ADD MAP MEMBER FUNCTION odwzoruj
RETURN NUMBER CASCADE INCLUDING TABLE DATA;
```

- Dodanie implementacji metody do definicji typu obiektowego

```
CREATE OR REPLACE TYPE BODY Pracownik AS
  MEMBER FUNCTION wiek RETURN NUMBER IS
  BEGIN
    RETURN EXTRACT (YEAR FROM CURRENT_DATE) –
           EXTRACT (YEAR FROM data_ur);
  END wiek;
  MEMBER PROCEDURE podwyzka(p_kwota NUMBER) IS
  BEGIN
    pensja := pensja + p_kwota;
  END podwyzka;
  MAP MEMBER FUNCTION odwzoruj RETURN NUMBER IS
  BEGIN
    RETURN ROUND(pensja,-3) + wiek();
  END odwzoruj;
END;
```

Wykorzystanie metod porównujących

- Zalecenia
  - typ obiektowy może mieć tylko jedną metodę porównującą (porządkującą lub odwzorowującą)
  - metody odwzorowujące są bardziej efektywne, metody porządkujące są bardziej elastyczne
  - metody porównujące mogą być wywoływane jawnie lub niejawnie

```
INSERT INTO PracownicyObjTab VALUES
  (NEW Pracownik('Nowak',2000,'ADIUNKT',DATE '1961-02-15'));
INSERT INTO PracownicyObjTab VALUES
  (NEW Pracownik('Janiak',1800,'ASYSTENT',DATE '1973-12-02'));

SELECT p.nazwisko, p.pensja, p.wiek()
FROM PracownicyObjTab p
ORDER BY VALUE(p);

SELECT * FROM PracownicyObjTab p
WHERE VALUE(p) > ( SELECT VALUE(r) FROM PracownicyObjTab r
                   WHERE r.nazwisko = 'Janiak' );
```

Konstruktor

- Konstruktor to specjalna metoda tworząca nowe obiekty
  - nazwa konstruktora jest taka sama jak nazwa typu obiektowego
  - użytkownik może definiować własne konstruktory, może także przesłonić domyślny konstruktor atrybutowy
  - konstruktory ułatwiają inicjalizację i umożliwiają ewolucję obiektów

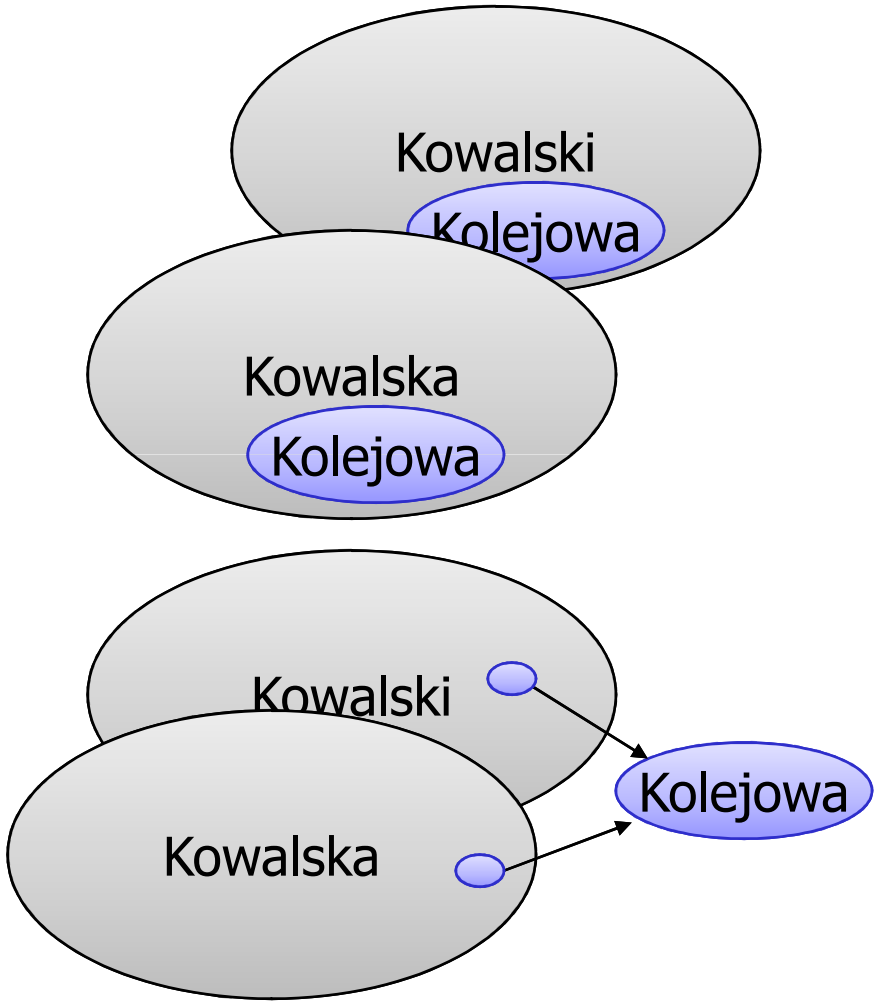
```
ALTER TYPE Pracownik REPLACE AS OBJECT (
  nazwisko VARCHAR2(20),
  pensja   NUMBER(6,2),
  etat     VARCHAR2(15),
  data_ur  DATE,
  MEMBER FUNCTION wiek RETURN NUMBER,
  MEMBER PROCEDURE podwyzka(p_kwota NUMBER),
  MAP MEMBER FUNCTION odwzoruj RETURN NUMBER,
  CONSTRUCTOR FUNCTION Pracownik(p_nazwisko VARCHAR2)
    RETURN SELF AS RESULT );
```

Implementacja konstruktora

```
CREATE OR REPLACE TYPE BODY Pracownik AS
  MEMBER FUNCTION wiek RETURN NUMBER IS
  BEGIN
    RETURN EXTRACT (YEAR FROM CURRENT_DATE) –
           EXTRACT (YEAR FROM data_ur);
  END wiek;
  MEMBER PROCEDURE podwyzka(p_kwota NUMBER) IS
  BEGIN
    pensja := pensja + p_kwota;
  END podwyzka;
  MAP MEMBER FUNCTION odwzoruj RETURN NUMBER IS
  BEGIN
    RETURN ROUND(pensja,-3) + wiek();
  END odwzoruj;

  CONSTRUCTOR FUNCTION Pracownik(p_nazwisko VARCHAR2)
    RETURN SELF AS RESULT IS
  BEGIN
    SELF.nazwisko := p_nazwisko; SELF.pensja := 1000;
    SELF.etat := null; SELF.data_ur := null;
    RETURN;
  END;
END;
```

Współdzielenie i zagnieżdżanie obiektów




```
CREATE TYPE TAdres AS OBJECT (
  ulica VARCHAR2(15),
  dom NUMBER(4),
  mieszkanie NUMBER(3));
```

```
CREATE TYPE Osoba AS OBJECT  (
  nazwisko VARCHAR2(20),
  imie VARCHAR2(15),
  adres TAdres);
```

```
CREATE TYPE Osoba AS OBJECT  (
  nazwisko VARCHAR2(20),
  imie VARCHAR2(15),
  adres REF TAdres);
```

Wiązanie obiektów współdzielonych

- Utworzenie tabel obiektowych

```
CREATE TABLE AdresyObjTab OF TAdres;
CREATE TABLE OsobyObjTab OF Osoba;
```

```
ALTER TABLE OsobyObjTab ADD
SCOPE FOR(adres) IS AdresyObjTab;
```

aby zawęzić zakres referencji tabela OsobyObjTab musi być pusta

- Wstawienie obiektów

```
INSERT INTO AdresyObjTab VALUES
(NEW TAdres('Kolejowa',2,18));

INSERT INTO OsobyObjTab VALUES
(NEW Osoba('Kowalska','Anna',null));
INSERT INTO OsobyObjTab VALUES
(NEW Osoba('Kowalski','Jan',null));

UPDATE OsobyObjTab o
SET o.adres = (
  SELECT REF(a) FROM AdresyObjTab a
  WHERE a.ulica = 'Kolejowa' );
```

Wykorzystanie referencji do nawigacji

- Referencje można wykorzystać w następujący sposób
  - nawigacja jawna przez wywołanie funkcji **DEREF( )**
  - nawigacja niejawna przez notację kropkową
  - operacja połączenia tabel przez porównanie referencji

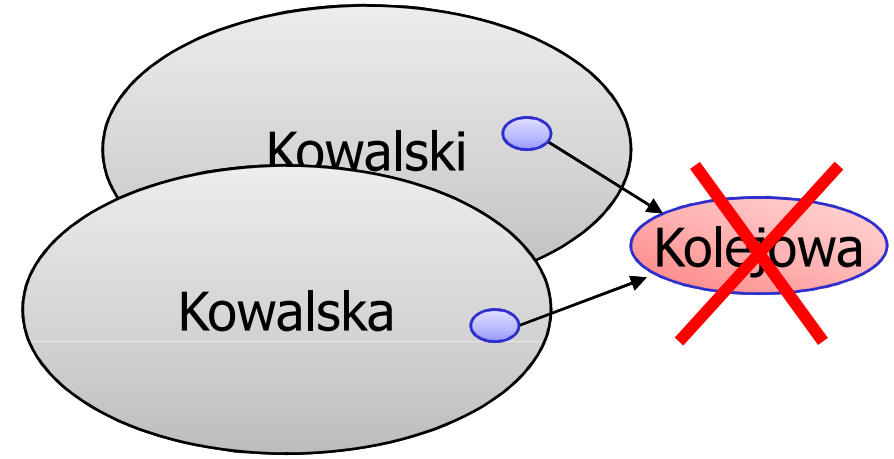
```
SELECT o.imie, o.nazwisko, DEREF(o.adres)
FROM OsobyObjTab o;
```

```
SELECT o.imie, o.nazwisko, o.adres.ulica, o.adres.dom
FROM OsobyObjTab o;
```

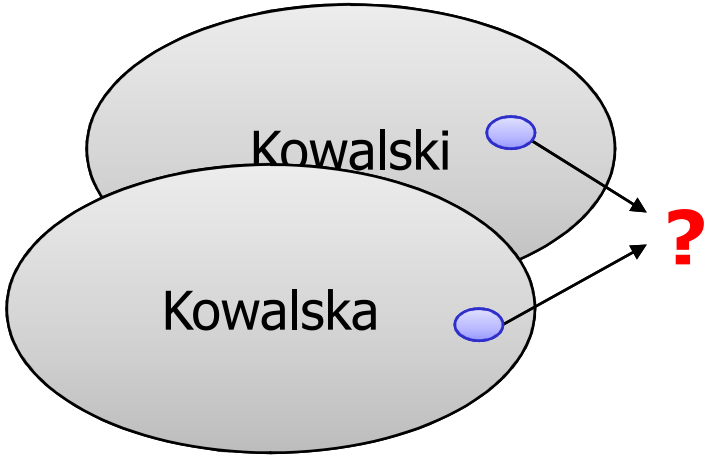
```
SELECT o.imie, o.nazwisko, a.ulica, a.dom, a.mieszkanie
FROM OsobyObjTab o JOIN AdresyObjTab a
  ON o.adres = REF(a);
```

Wiszące referencje

- Usunięcie obiektu nie usuwa referencji do obiektu




```
DELETE FROM AdresyObjTab a
WHERE a.ulica = 'Kolejowa';
```




```
SELECT * FROM OsobyObjTab o
WHERE o.adres IS NULL;
```

```
SELECT * FROM OsobyObjTab o
WHERE o.adres IS DANGLING;
```

Ograniczenia integralnościowe i słownik bazy danych

- Definiowanie ograniczeń integralnościowych jest możliwe tylko dla tabel obiektowych

```
ALTER TABLE OsobyObjTab
ADD PRIMARY KEY(imie,nazwisko);
```

```
ALTER TABLE PracownicyObjTab
ADD CONSTRAINT niezerowa_pensja CHECK (pensja > 0);
```

```
ALTER TABLE PracownicyObjTab
ADD CONSTRAINT niepusty_etat etat NOT NULL;
```

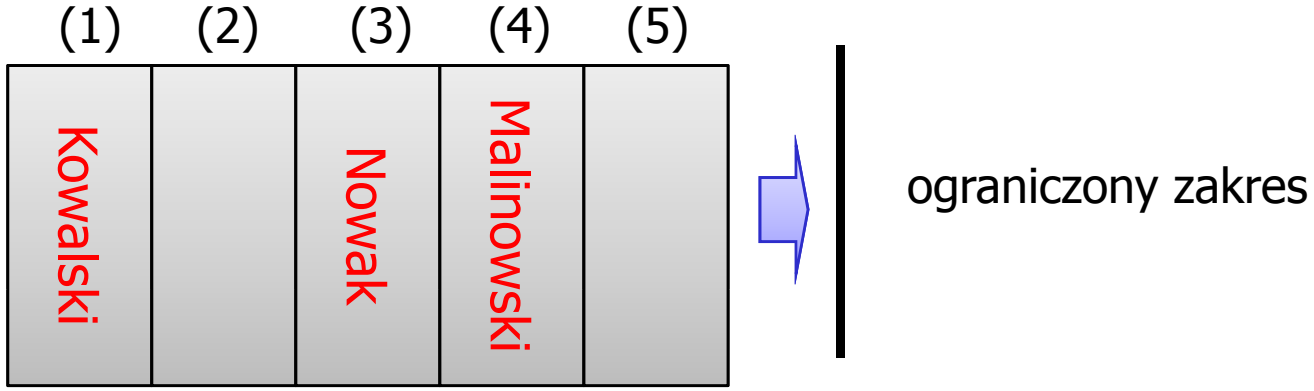
- Perspektywy **USER_TYPES** i **USER_SOURCE** słownika

```
SELECT type_name,typecode,attributes,methods FROM user_types;

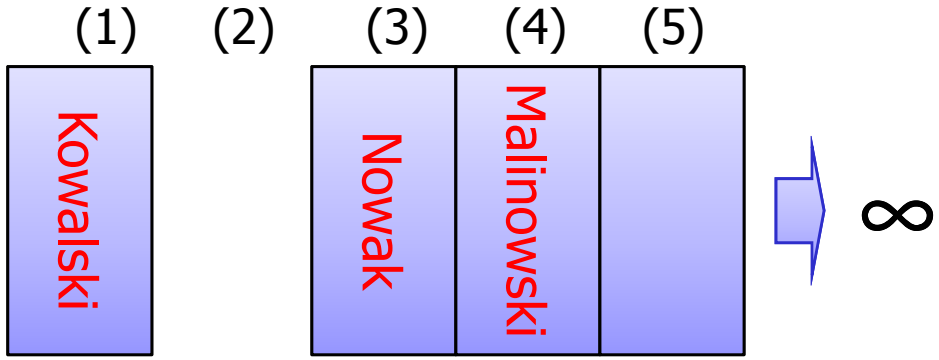
SELECT text FROM user_source WHERE name = 'OSOBA';
```

Kolekcje

- Kolekcje to zbiory obiektów
  - tablica o zmiennym rozmiarze (ang. *varray*)



  - tablica zagnieżdżona (ang. *nested table*)



Porównanie typów kolekcji

własność	VARRAY	NESTED TABLE
maksymalny rozmiar	tak	nie
usuwanie elementów ze środka kolekcji	nie	tak
składowanie w bazie danych	in-line	out-line
zachowanie fizycznego porządku	tak	nie
manipulowanie na pojedynczych elementach kolekcji w SQL	nie	tak

Tworzenie kolekcji

- Kolekcje mogą być tworzone:
  - na poziomie SQL: tworzone na stałe w bazie danych
  - na poziomie PL/SQL: lokalne deklaracje na potrzeby programu

```
CREATE TYPE Telefony AS VARRAY(10) OF VARCHAR(15);
```

```
DECLARE
  TYPE Telefony IS VARRAY(10) OF VARCHAR(15);
  moi_znajomi Telefony;
BEGIN
  ...
```

kolekcja typu tabela o zmiennym rozmiarze utworzona z poziomu SQL i PL/SQL

```
CREATE TYPE Zespol AS TABLE OF Pracownik;
```

```
DECLARE
  TYPE Zespol IS TABLE OF Pracownik;
  bazy_danych Zespol;
BEGIN
  ...
```

kolekcja typu tabela zagnieżdżona utworzona z poziomu SQL i PL/SQL

Metody kolekcji

metoda	opis
kolekcja(wartość,...)	konstruktor kolekcji, opcjonalnie wstawia wartości jako kolejne elementy kolekcji
EXTEND([n],[i])	rozszerza kolekcję o n pustych elementów, opcjonalnie wypełnia wartością i-tego elementu
TRIM([n])	usuwa n elementów od końca kolekcji
DELETE([n],[m])	usuwa wszystkie elementy kolekcji, n-ty element, lub elementy od n-tego do m-tego
NEXT(n), PRIOR(n)	zwraca indeks elementu następującego (poprzedzającego) elementu o indeksie n
EXISTS(n)	testuje istnienie elementu o indeksie n
FIRST, LAST	zwraca indeks pierwszego (ostatniego) elementu
LIMIT	zwraca maksymalny zakres kolekcji
COUNT	zwraca liczbę elementów kolekcji

Tabela o zmiennym rozmiarze w PL/SQL

```
DECLARE
  znajomi Telefony := Telefony();
BEGIN
  znajomi.EXTEND();
  znajomi(1) := 'Bolek – 1234567';
  znajomi.EXTEND(znajomi.LIMIT()-znajomi.COUNT(),1);

  FOR i IN 1..znajomi.COUNT() LOOP
    DBMS_OUTPUT.PUT_LINE(znajomi(i));
  END LOOP;

  znajomi.TRIM(4);
  znajomi(2) := 'Lolek – 9876543';
  znajomi(5) := 'Tola – 0102030';

  FOR i IN 1..znajomi.COUNT() LOOP
    DBMS_OUTPUT.PUT_LINE(znajomi(i));
  END LOOP;
END;
```

Tabela o zmiennym rozmiarze w SQL

- Jeśli kolekcja jest tabelą o zmiennym rozmiarze
  - nie można manipulować pojedynczymi elementami w SQL
  - elementy zachowują fizyczny porządek

```
CREATE TABLE znajomi (imie VARCHAR(100), kontakt Telefony);

INSERT INTO znajomi VALUES
  ('Bolek', Telefony('1234567','2345678'));
INSERT INTO znajomi VALUES
  ('Lolek', Telefony('1111111','2222222'));

SELECT kontakt FROM znajomi WHERE imie = 'Bolek';

UPDATE znajomi
SET kontakt = Telefony('1234567','2345678','3456789')
WHERE imie = 'Bolek';
```

Tabela zagnieżdżona w PL/SQL

```
DECLARE
  bazy_danych Zespol := Zespol();
  i NUMBER;
BEGIN
  bazy_danych.EXTEND(1);
  bazy_danych(1) := NEW Pracownik('Bolek');
  bazy_danych.EXTEND(5,1);
  bazy_danych.DELETE(2,3);

  FOR i IN 1..bazy_danych.LAST() LOOP
    IF bazy_danych.EXISTS(i) THEN
      DBMS_OUTPUT.PUT_LINE(bazy_danych(i).nazwisko);
    END IF;
  END LOOP;

  bazy_danych(4) := NEW Pracownik('Lolek');
  bazy_danych(5) := NEW Pracownik('Tola');

  i := bazy_danych.FIRST();
  WHILE (i IS NOT NULL) LOOP
    DBMS_OUTPUT.PUT_LINE(bazy_danych(i).nazwisko);
    i := bazy_danych.NEXT(i);
  END LOOP;
END;
```

Tabela zagnieżdżona w SQL

- Jeśli kolekcja jest tabelą zagnieżdżoną
  - można manipulować pojedynczymi elementami w SQL
  - należy wskazać tabelę out-line do przechowywania kolekcji

```
CREATE TABLE projekty(nazwa VARCHAR(100), wykonawcy Zespol)
NESTED TABLE wykonawcy STORE AS wykonawcy_projektu;

INSERT INTO projekty VALUES ('Archiwum X',
  NEW Zespol(NEW Pracownik('Bolek'), NEW Pracownik('Lolek')));

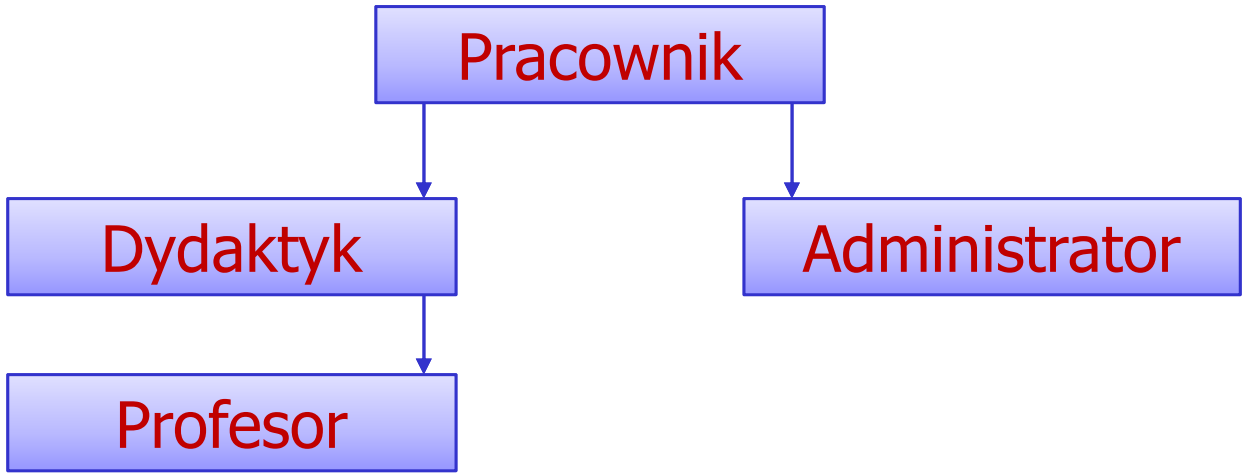
SELECT * FROM TABLE(
  SELECT wykonawcy FROM projekty WHERE nazwa = 'Archiwum X');

INSERT INTO TABLE (
  SELECT wykonawcy FROM projekty WHERE nazwa = 'Archiwum X')
VALUES (NEW Pracownik('Tola'));

UPDATE TABLE (
  SELECT wykonawcy FROM projekty WHERE nazwa = 'Archiwum X')
SET etat = 'KIEROWNIK', pensja = 5000
WHERE nazwisko = 'Bolek';
```

Dziedziczenie

- Dziedziczenie polega na definiowaniu nowego typu obiektowego w oparciu o istniejący typ obiektowy
  - nowy typ stanowi podtyp (specjalizację) swojego nadtypu (przodka)
  - podtyp dziedziczy wszystkie składowe i metody **`MEMBER`** i **`STATIC`**
  - podtyp może dodawać nowe składowe i przesłaniać metody
  - każdy podtyp może mieć tylko jeden nadtyp
  - podtyp może dziedziczyć tylko z nadtypu który został zadeklarowany jako **`NOT FINAL`** (uwaga: domyślnie każdy typ jest **`FINAL`**)
  - metody porządkujące mogą się pojawić tylko w korzeniu hierarchii



Dziedziczenie

```
ALTER TYPE Pracownik NOT FINAL CASCADE;
```

przesłonięcie metody

przeciążenie metody

```
CREATE TYPE Dydaktyk UNDER Pracownik (
  tytul VARCHAR2(10),
  OVERRIDING MEMBER FUNCTION wiek RETURN NUMBER,
  MEMBER FUNCTION wiek(l_data DATE) RETURN NUMBER);
```

```
CREATE TYPE BODY Dydaktyk AS
  OVERRIDING MEMBER FUNCTION wiek RETURN NUMBER IS
  BEGIN
    RETURN ROUND(MONTHS_BETWEEN(CURRENT_DATE, data_ur));
  END wiek;

  MEMBER FUNCTION wiek(l_data DATE) RETURN NUMBER IS
  BEGIN
    RETURN EXTRACT(YEAR FROM l_data) –
           EXTRACT(YEAR FROM data_ur);
  END wiek;
END;
```

Polimorfizm

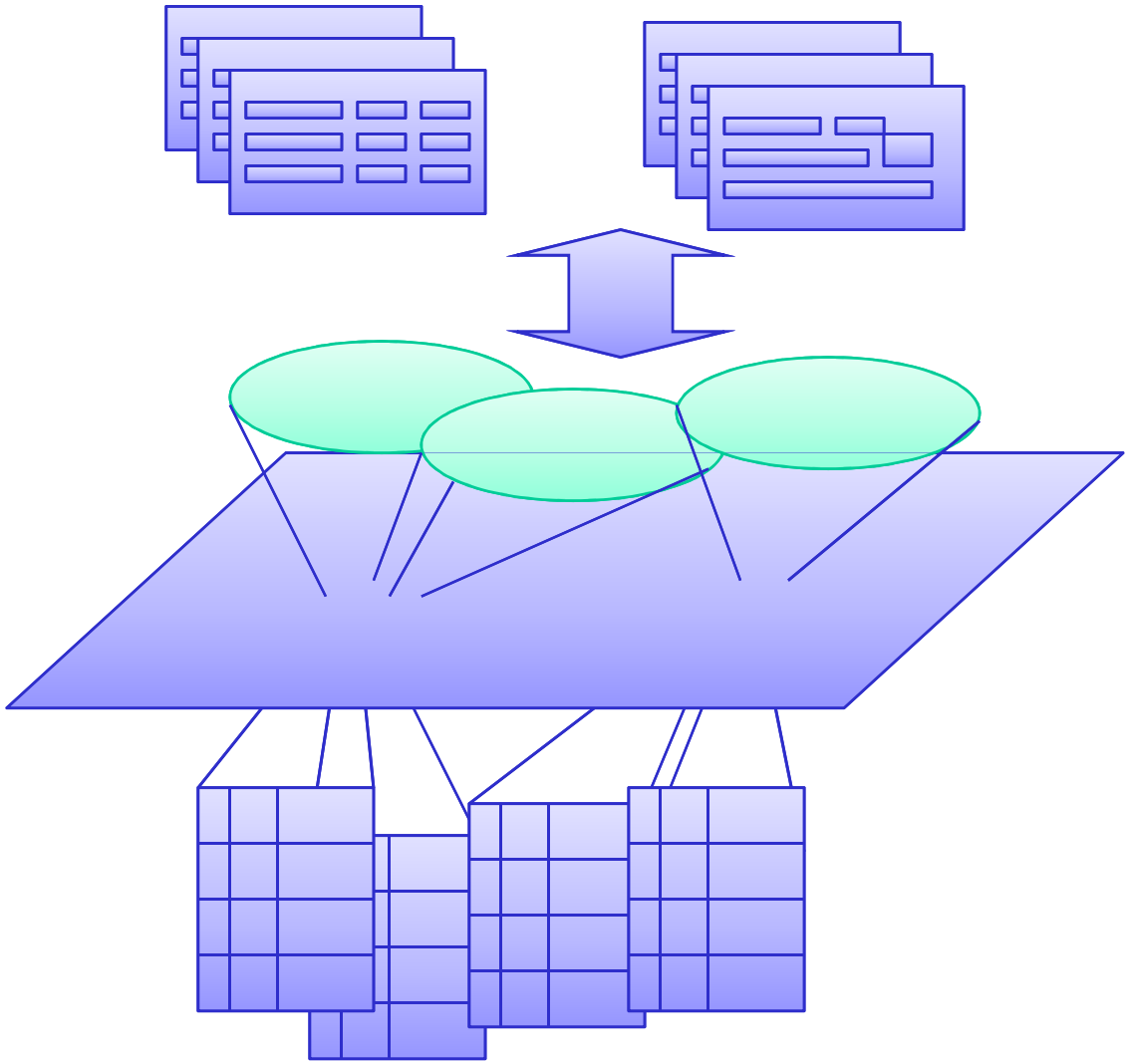
- Polimorfizm powoduje, że obiekty podtypu mogą zachowywać się jak obiekty swojego nadtypu
  - zamiast obiektu nadtypu można wykorzystać obiekt podtypu
  - zamiast obiektu podtypu można wymusić wykorzystanie obiektu nadtypu przez użycie operatora **TREAT**
  - wybór wersji metody przeciążonej następuje w momencie wykonania programu (metody są wirtualne)

```
CREATE TABLE pracownicyIdydaktycyObjTab OF PRACOWNIK;

INSERT INTO pracownicyIdydaktycyObjTab VALUES
(NEW Pracownik('Bolek',2000,'ASYSTENT',DATE '1969-09-03'));
INSERT INTO pracownicyIdydaktycyObjTab VALUES
(NEW Dydaktyk('Lolek',5000,'PROF.',DATE '1949-11-13','dr.'));

SELECT p.nazwisko, p.wiek() FROM pracownicyIdydaktycyObjTab p;
```

Perspektywy obiektowe

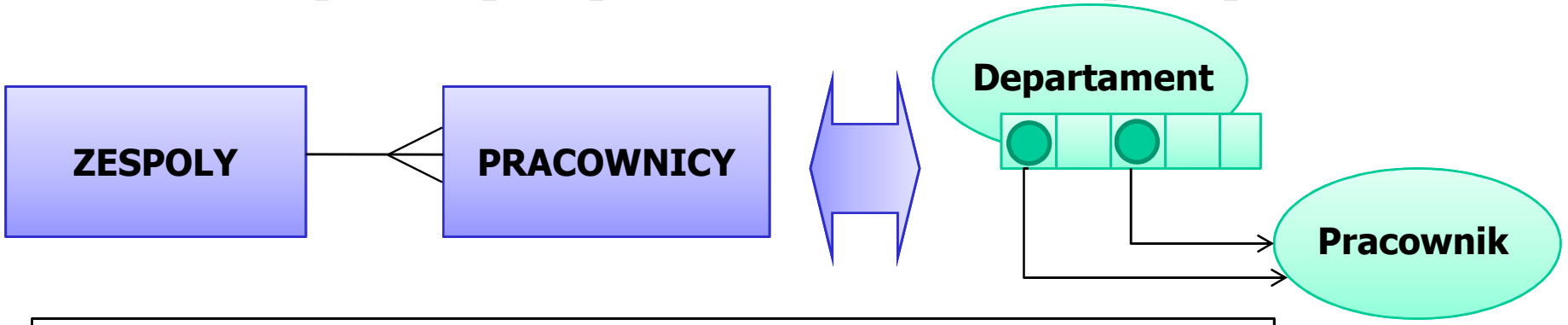


aplikacje

perspektywy obiektowe

świat relacyjny

Perspektywy obiektowe - przykład




```
CREATE TYPE PracownicyTab AS TABLE OF Pracownik;

CREATE TYPE Departament AS OBJECT (
  nazwa VARCHAR2(100),
  adres VARCHAR2(100),
  pracownicy PracownicyTab,
  MEMBER FUNCTION iluPracownikow RETURN NUMBER );

CREATE OR REPLACE TYPE BODY Departament AS
  MEMBER FUNCTION iluPracownikow RETURN NUMBER IS
  BEGIN
    RETURN pracownicy.COUNT();
  END iluPracownikow;
END;
```

Perspektywy obiektowe - przykład

- Utworzenie perspektywy obiektowej

```
CREATE OR REPLACE VIEW DepartamentyObjView OF Departament
WITH OBJECT OID(nazwa) AS
SELECT z.nazwa, z.adres, CAST( MULTISET(
  SELECT NEW Pracownik(nazwisko,placa_pod,etat,zatrudniony)
  FROM pracownicy WHERE id_zesp = z.id_zesp) AS PracownicyTab)
FROM zespoly z;
```

- Możliwe zapytania

```
SELECT * FROM DepartamentyObjView;

SELECT d.nazwa, d.adres, d.iluPracownikow()
FROM DepartamentyObjView d;

SELECT nazwisko, pensja, etat, data_ur
FROM TABLE( SELECT pracownicy
            FROM DepartamentyObjView
            WHERE nazwa = 'SYSTEMY ROZPROSZONE' );
```